```
package A01_Stack;

public class Stack<T>
{
      private Node<T> first;

    private int count = 0;

    /**
     * Oberstes Element entfernen und zur?ckliefern.
     * Existiert kein Element, wird eine Exception ausgel?st.
     * @throws StackEmptyException
     */
    public T pop() throws StackEmptyException {
      if (count == 0) {
            throw new StackEmptyException();
      }
      T value = first.getData();
      first = first.getNext();
      count--;
      return value;
    }

    /**
     * ?bergebenen Integer auf Stack (als oberstes Element) speichern.
     * @param i Zahl
     */
    public void push(T i) {
      Node<T> n = new Node<T>(i);
      n.setNext(first);
      first = n;
      count++;
    }

    /**
     * Liefert die Anzahl der Elemente im Stack
     * @return
     */
    public int getCount() {
      return count;
    }
}

package A02_Queue;

public class Queue<T>
{
    private Node<T> first;

    private Node<T> last;

    int count = 0;

    public T dequeue() throws QueueEmptyException {
      if (count == 0) {
            throw new QueueEmptyException();
      }
      T value = first.getData();
      first = first.getNext();
```

```
      count--;
      if (count == 0) {
            last = null;
      }
      return value;
    }


    public void enqueue(T i) {
      Node<T> n = new Node<T>(i);
      if (count == 0) {
            first = n;
      }
      else {
            last.setNext(n);
      }
            last = n;
      count++;
    }


    public int getCount() {
      return count;
    }
}
```

```
package A03_DoubleLinkedList;


public class DoubleLinkedList<T>
{

      private Node<T> first;

    private Node<T> last;

    private Node<T> current;

    /**
     * Einf?gen einer neuen Ausrede
     * @param a Ausrede
     */
    public void add(T a) {
        Node<T> n = new Node<T>(a);
        if (last == null) {     // Liste ist noch leer
            first = n;
            last = n;
        }
        else {                  // Liste enth?lt bereits Elemente
            last.setNext(n);
            n.setPrevious(last);
            last = n;
        }
    }

    /**
     * Internen Zeiger f?r next() zur?cksetzen
     */
    public void reset() {
```

```
    current = first;
}

/**
 * analog zur Funktion reset()
 */
public void resetToLast() {
  current = last;
}

/**
 * Liefert erste Node der Liste retour oder null, wenn Liste leer
 * @return Node|null
 */
public Node<T> getFirst() {
  return first;
}

/**
 * Liefert letzte Node der Liste retour oder null, wenn Liste leer
 * @return Node|null
 */
public Node<T> getLast() {
  return last;
}

/**
 * Gibt aktuelle Ausrede zur?ck und setzt internen Zeiger weiter.
 * Falls current nicht gesetzt, wird null retourniert.
 * @return Ausrede|null
 */
public T next() {
  if (current == null) {
        return null;
  }
  T a = current.getData();
  current = current.getNext();
  return a;
}

/**
 * analog zur Funktion next()
 * @return Ausrede|null
 */
public T previous() {
  if (current == null) {
        return null;
  }
  T a = current.getData();
  current = current.getPrevious();
  return a;
}

/**
 * Current-Pointer auf n?chste Ausrede setzen (aber nicht auslesen).
 * Ignoriert still, dass current nicht gesetzt ist.
 */
public void moveNext() {
  if (current != null)
```

```
        current = current.getNext();
}

/**
 * Analog zur Funktion moveNext()
 */
public void movePrevious() {
  if (current != null)
        current = current.getPrevious();
}

/**
 * Retourniert aktuelle (current) Ausrede, ohne Zeiger zu ?ndern
 * @return Ausrede
 * @throws CurrentNotSetException
 */
public T getCurrent() throws CurrentNotSetException {
  if (current == null) {
        throw new CurrentNotSetException();
  }
  return current.getData();
}

/**
 * Gibt Ausrede an bestimmter Position zur?ck
 * @param pos Position, Nummerierung startet mit 1
 * @return Ausrede|null
 */
public T get(int pos) {
    Node<T> n = first;
    int cnt = 1;
    while (cnt != pos && n != null) {
        n = n.getNext();
        cnt++;
    }
    return (n != null) ? n.getData() : null;
}

/**
 * Entfernen des Elements an der angegebenen Position
 * @param pos
 */
public void remove(int pos) {
    Node<T> n = first;
    int cnt = 1;
    while (cnt != pos && n != null) {   // Element suchen
        n = n.getNext();
        cnt++;
    }
    if (n == null) {    // keine Node an dieser Position
        return;
    }
    removeNode(n);
    if (current == n) {
        current = null;
    }
}

/**
```

```
 * Entfernt das aktuelle Element.
 * Als neues aktuelles Element wird der Nachfolger gesetzt oder
 * (falls kein Nachfolger) das vorhergehende Element
 * @throws CurrentNotSetException
 */
public void removeCurrent() throws CurrentNotSetException {
  if (current == null) {
        throw new CurrentNotSetException();
  }
        removeNode(current);
        if (current.getNext() != null) {
              current = current.getNext();
        }
        else {
              current = current.getPrevious();
        }
}

/**
 * Hilfsfunktion, die von remove() und removeCurrent() verwendet wird
 * @param n Zu l?schende Node
 */
private void removeNode(Node<T> n) {
    if (n == first) {
        first = n.getNext();
        if (first != null) {
              first.setPrevious(null);
        }
    }
    else if (n == last) {
        last = n.getPrevious();
        if (last != null) {
              last.setNext(null);
        }
    }
    else {
        n.getPrevious().setNext(n.getNext());
        n.getNext().setPrevious(n.getPrevious());
    }
}


/**
 * Die Methode f?gt die ?bergebene Ausrede nach der aktuellen (current) ein
 * und setzt dann die neu eingef?gte Ausrede als aktuelle (current) Ausrede.
 * @throws CurrentNotSetException
 */
public void insertAfterCurrentAndMove(T a) throws CurrentNotSetException {
  Node<T> n = new Node<T>(a);
  if (current == null) {
        throw new CurrentNotSetException();
  }
  else if (current == last) {
        last.setNext(n);
        n.setPrevious(last);
        last = n;
  }
  else {
        n.setNext(current.getNext());
```

```
            n.setPrevious(current);
            current.getNext().setPrevious(n);
            current.setNext(n);
      }
            current = n;
    }
}

package doublell;

public class DoubleLinkedList {

      private Node first = null;
      private Node last = null;
      private int count = 0;

      private Node current = null;

      public int getCount() {
            return count;
      }

      public void moveFirst() {
            current = first;
      }

      public void moveLast() {
            current = last;
      }

      public boolean moveNext() {
            if (current == last) {
            // if (current.getNext() == null) {
                  return false;
            }
            current = current.getNext();
            return true;
      }

      public void deleteCurrent() {

            if (count == 0) return;

            if (count == 1) {
                  first = null;
                  last = null;
                  current = null;
            }
            else if (current == first) {
                  current = first.getNext();
                  current.setPrevious(null);
                  first = current;
            }
            else if (current == last) {
                  current = last.getPrevious();
                  current.setNext(null);
                  last = current;
            }
            else {
```

```
			Node a = current.getPrevious();
			Node b = current.getNext();

			a.setNext(b);
			b.setPrevious(a);
			current = b;
		}
		count--;



	}


	public boolean movePrevious() {
		if (current == first) {
		//if (current.getPrevious() == null) {
			return false;
		}
		current = current.getPrevious();
		return true;
	}

	public int getCurrent() {
		if (current == null) {
			System.err.println("Liste ist leer!");
			return -1;
		}
		return current.getData();
	}

	// TODO
	private void insert(int item) {
		Node n = new Node();
		n.setData(item);

		// TODO: Schreiben Sie hier ihren Code
		// n soll nach current eingef◆gt werden und n wird das neue current

		if (count != 0) {
			if (current.getNext() != null) {
				n.setNext(current.getNext());
				current.getNext().setPrevious(n);
			} else {
				last = n;
			}

			n.setPrevious(current);
			current.setNext(n);
		} else {
			first = n;
			last = n;
		}

		current = n;
		count++;
	}

	private void _insertBefore(int item) {
```

```
	Node n = new Node();
	n.setData(item);

	// TODO: Schreiben Sie hier ihren Code
	// n soll nach current eingef◆gt werden und n wird das neue current

	if (count != 0) {
		if (current.getPrevious() != null) {
			n.setPrevious(current.getPrevious());
			current.getPrevious().setNext(n);
		} else {
			first = n;
		}

		n.setNext(current);
		current.setPrevious(n);
	} else {
		first = n;
		last = n;
	}
	count++;
}

// TODO
public void insertSorted(int item) {

	// TODO: Schreiben Sie hier ihren Code
	if (count == 0) {
		insert(item);
		return;
	}

	moveFirst();
	_place(item);
}

private void _place(int item) {
	int data = current.getData();
	if (data < item) {
		if (current.getNext() == null)
			insert(item);
		else if (current.getNext().getData() > item) {
			moveNext();
			_place(item);
		}
		else if (current.getNext().getData() <= item)
			insert(item);
	} else {
		_insertBefore(item);
	}
}


public int getAt(int pos) {
	if (!(pos >= 0 && pos < count)) {
		System.err.println("Position ist nicht vorhanden: " + pos);
		return -1;
	}
```

```
            Node current = first;

            for(int i=0; i<pos; i++) {
                  current = current.getNext();
            }

            return current.getData();
      }

}

package A04_TraverseTree;

import java.util.HashSet;
import java.util.Set;


public class W?rterbuch {

      /**
       * Wurzel des Baums (Startknoten)
       */
      private Wort root;

      public Wort getRoot() {
            return root;
      }

      /**
       * Z?hlt alle W?rter des Teilbaums ab einem bestimmten Wort
       * @param w Wort
       * @return Zahl der W?rter (=Anzahl der Elemente)
       */
      public int countWordsInSubTree(Wort w) {

            if (w == null) {
                  return 0;
            }
            // rekursiv aufsummieren
            int result = 1 + countWordsInSubTree(w.getLeft()) +
countWordsInSubTree(w.getRight());
            return result;
      }

      public Set<String> getWordsWithPrefix(String prefix) {
            return getWordsWithPrefix(root, prefix);
      }

      private Set<String> getWordsWithPrefix(Wort w, String prefix) {
            HashSet<String> ll = new HashSet<String>();
            if (w == null) {
                  return ll;
            }
            ll.addAll(getWordsWithPrefix(w.getLeft(), prefix));
            if (w.getWort().startsWith(prefix)) {
                  ll.add(w.getWort());
            }
            ll.addAll(getWordsWithPrefix(w.getRight(), prefix));
            return ll;
```

```
	}


	/**
	 * Neues Wort hinzuf?gen
	 * @param wort Hinzuzuf?gendes Wort
	 */
	public void add(String wort) {
		Wort neu = new Wort(wort);
		if (root == null) {		// Fall 1: Baum ist leer
			root = neu;
			return;
		}
		Wort w = root;			// Fall 2: Baum ist nicht leer
		while (true) {
			int vgl = wort.compareTo(w.getWort());
			if (vgl < 0) {		// Neues Wort ist lexikographisch
kleiner
				if (w.getLeft() == null) {
					w.setLeft(neu);
					neu.setParent(w);
					return;
				}
				w = w.getLeft();
			}
			else if (vgl > 0) {	// Neues Wort ist lexikographisch
gr??er
				if (w.getRight() == null) {
					w.setRight(neu);
					neu.setParent(w);
					return;
				}
				w = w.getRight();
			}
			else {				// Neues Wort ist
lexikographisch gleich
				return;
			}
		}
	}

	public Wort find(String s) {
		return find(root, s);
	}

	private Wort find(Wort current, String s) {
		if (current == null) {
			return null;
		}
		int vgl = s.compareTo(current.getWort());
		if (vgl == 0) {		// Gefunden
			return current;
		}
		else if (vgl < 0) {	// Links
			return find(current.getLeft(), s);
		}
		else {				// Rechts
			return find(current.getRight(), s);
		}
```

```
	}

}

package A05_Breitensuche;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.List;
import java.util.Queue;

public class Breitensuche extends BaseTree<Integer> {

	@Override
	protected int compare(Integer a, Integer b) {
		return a.compareTo(b);
	}

	/**
	 * Liefert Knoten des Baums ausgehend von Start in Reihenfolge der
Breitensuche zur?ck
	 * @param start Startknoten f?r Teilbaum
	 * @return Liste der Knoten in Breitenfolge
	 */
	public List<Integer> getBreadthFirstOrder(Node<Integer> start) {

		// Breitensuche verwendet eine Queue
		Queue<Node<Integer>> qi = new ArrayDeque<Node<Integer>>();
		List<Integer> li = new ArrayList<Integer>();
		Node<Integer> node = start;
		while (node != null) {
			li.add(node.getValue());
			if (node.getLeft() != null) {
				qi.add(node.getLeft());
			}
			if (node.getRight() != null) {
				qi.add(node.getRight());
			}
			node = qi.poll();
		}
		return li;
	}

	/**
	 * Liefert Knoten des Baums ausgehend von Start in Reihenfolge der
Breitensuche zur?ck,
	 * allerdings nur jene Knoten, die in der angegebenen Ebene liegen (Start hat
Ebene=1)
	 * @param start Startknoten f?r Teilbaum
	 * @param level Nur Knoten dieser Ebene ausgeben
	 * @return Liste aller Knoten
	 */
	public List<Integer> getBreadthFirstOrderForLevel(Node<Integer> start, int
level) {

		/**
		 * Wir m?ssen uns zu jeder Node auch noch deren Ebene merken.
		 * Variante 1: Klasse Node erweitern
		 * Variante 2: (hier verwendet): Wrapper-Klasse, die Node und Level
```

```
speichert
             */
            class NodePlusLevel {
                  protected Node<Integer> node;
                  protected int level;
                  public NodePlusLevel(Node<Integer> node, int level) {
                        this.node = node;
                        this.level = level;
                  }
            }

            // Breitensuche verwendet eine Queue
            Queue<NodePlusLevel> qi = new ArrayDeque<NodePlusLevel>();
            List<Integer> li = new ArrayList<Integer>();
            NodePlusLevel node = new NodePlusLevel(start, 1);    // Start ist in
Ebene 1
            while (node != null && node.node != null) {
                  if (node.level == level) {                                    // Nur
Werte der passenden Ebene dem Ergebnis hinzuf?gen
                        li.add(node.node.getValue());
                  }
                  if (node.level < level) {                                     //
Sobald Ebene >= Zielebene, brauchen wir keine Kinder mehr hinzuf?gen
                        if (node.node.getLeft() != null) {
                              qi.add(new NodePlusLevel(node.node.getLeft(),
node.level+1));
                        }
                        if (node.node.getRight() != null) {
                              qi.add(new NodePlusLevel(node.node.getRight(),
node.level+1));
                        }
                  }
                  node = qi.poll();
            }
            return li;
      }

}

package A06_Tiefensuche;

import java.util.ArrayList;
import java.util.List;

import A05_Breitensuche.BaseTree;
import A05_Breitensuche.Node;

public class Tiefensuche extends BaseTree<Film> {

      @Override
      /**
       * Sortierkriterium im Baum: L?nge des Films
       */
      protected int compare(Film a, Film b) {
            if (a.getL?nge() < b.getL?nge()) {
                  return -1;
            }
            if (a.getL?nge() > b.getL?nge()) {
                  return 1;
```

```
            }
            return 0;
      }

      /**
       * Retourniert die Titelliste der Film-Knoten des Teilbaums in symmetrischer
Folge (engl. in-order, d.h. links-Knoten-rechts)
       * @param node Wurzelknoten des Teilbaums
       * @return Liste der Titel in symmetrischer Reihenfolge
       */
      public List<String> getNodesInOrder(Node<Film> node) {

            ArrayList<String> als = new ArrayList<String>();
            if (node != null) {
                  als.addAll(getNodesInOrder(node.getLeft()));
                  als.add(node.getValue().getTitel());
                  als.addAll(getNodesInOrder(node.getRight()));
            }
            return als;
      }

      /**
       * Retourniert Titelliste jener Filme, deren L?nge zwischen min und max
liegt, in Hauptreihenfolge (engl. pre-order, d.h. Knoten-links-rechts)
       * @param min Minimale L?nge des Spielfilms
       * @param max Maximale L?nge des Spielfilms
       * @return Liste der Filmtitel in Hauptreihenfolge
       */
      public List<String> getMinMaxPreOrder(double min, double max) {
            return getMinMaxPreOrder(root, min, max);
      }

      /**
       * Rekursive Hilfsfunktion, Beschreibung siehe oben
       */
      private List<String> getMinMaxPreOrder(Node<Film> node, double min, double
max) {
            ArrayList<String> als = new ArrayList<String>();
            if (node == null) {
                  return als;
            }
            double l?nge = node.getValue().getL?nge();
            if (l?nge >= min && l?nge < max) {
                  als.add(node.getValue().getTitel());
            }
            if (l?nge >= min) {
                  als.addAll(getMinMaxPreOrder(node.getLeft(), min, max));
            }
            if (l?nge < max) {
                  als.addAll(getMinMaxPreOrder(node.getRight(), min, max));
            }
            return als;
      }

}

package A07_BubbleSort;
```

```
public class BubbleSort implements PersonenSort {

	/**
	 * Sortier-Funktion
	 */
	public void sort(Person[] personen) {
		boolean exchanged = true;
		int count = 0;

		// Sortieren, bis keine Vertauschungen mehr stattfinden
		while (exchanged) {
			exchanged = false;
			// Mit jedem Durchlauf wird das Array mehr sortiert (letzte
Elemente passen)
			for (int i = 0; i < (personen.length - 1 - count); i++) {
				Person a = personen[i];
				Person b = personen[i+1];
				if (a.compareTo(b) > 0) {		// Tauschen
					personen[i] = b;
					personen[i+1] = a;
					exchanged = true;
				}
			}
			count++;
		}
	}

}

package A08_GraphZusammenh◆ngend;

import java.util.List;


import basisAlgorithmen.Graph;
import basisAlgorithmen.WeightedEdge;

public class ConnectedComponents {

	private boolean[] visited;
	/**
	 * Retourniert die Anzahl der zusammenh?ngenden Komponenten eines Graphen
	 * @param g zu pr?fender Graph
	 * @return Anzahl der Komponenten
	 */
	public int getNumberOfComponents(Graph g) {
		int n = g.numVertices();
		visited = new boolean[n];
		int count = 0;
		for (int i=0; i < n; i++) {
			if (visited[i]) {
				continue;
			}
			count++;
			depthSearch(g, i);
		}
		return count;
	}
```

```
	/**
	 * Rekursive Tiefensuche im Graphen, die Knoten in visited[] als besucht
markiert
	 * @param g zu pr?fender Graph
	 * @param v Startknoten
	 */
	private void depthSearch(Graph g, int v) {
		if (visited[v]) {
			return;
		}
		visited[v] = true;
		List<WeightedEdge> lwe = g.getEdges(v);
		for (WeightedEdge we : lwe) {
			depthSearch(g, we.to_vertex);
		}
	}

}

package A09_ZyklenTiefensuche;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Queue;
import java.util.concurrent.ArrayBlockingQueue;

import basisAlgorithmen.Graph;
import basisAlgorithmen.Vertex;
import basisAlgorithmen.WeightedEdge;

public class Zyklen {

	private Graph g;
	private boolean visited[];
	private HashMap<Integer, Integer> follow;


	public Zyklen(Graph g) {
		this.g = g;
	}

	/**
	 * Retourniert einen Zyklus eines Graphen, sofern einer existiert
	 * @param g zu pr?fender Graph
	 * @return Anzahl der Komponenten
	 */
	public List<Integer> getCycle() {
		int n = g.numVertices();
		visited = new boolean[n];
		follow = new HashMap<Integer, Integer>(n);
		for (int i=0; i < n; i++) {		// der Reihe nach mit allen Knoten
durchmachen
			if (visited[i]) {
				continue;
			}
			List<Integer> res = checkForCycle(i, -1);
			if (res != null) {			// Zyklus gefunden?
```

```
                        return res;
                  }
            }
            return null;                              // kein Zyklus gefunden
      }

      /**
       * Rekurisve Funktion zum Suchen des Zyklus
       * @param v    Aktueller Knoten
       * @param pred Vorg?nger
       * @return  Zyklus oder null
       */
      private List<Integer> checkForCycle(Integer v, Integer pred) {
            if (follow.containsKey(v)) {
                  return foundCycle(v);
            }
            if (visited[v]) {                  // Knoten bereits besucht
                  return null;
            }
            visited[v] = true;
            follow.put(pred, v);               // Nachfolger in Liste einf?gen
            List<WeightedEdge> lwe = g.getEdges(v);
            for (WeightedEdge we : lwe) { // alle Kanten durchgehen
                  if (we.to_vertex == pred && !g.isDirected()) { // ungerichteter
Graph: direkter Weg zur?ck
                        continue;
                  }
                  List<Integer> res = checkForCycle(we.to_vertex, v);  // Rekursion
                  if (res != null) {      // Zyklus gefunden
                        return res;
                  }
            }
            follow.remove(pred);    // Backtracking: Vorg?nger wieder entfernen
            return null;                  // kein Zyklus gefunden
      }

      /**
       * Generiert Liste des Weges aus follow-HashMap
       * @param start Startpunkt des Zyklus
       * @return Liste der Knoten des Zyklus in passender Besuchsreihenfolge
       */
      private List<Integer> foundCycle(Integer start) {
            List<Integer> cycle = new ArrayList<Integer>();
            Integer cur = start;
            do {
                  cycle.add(cur);
                  cur = follow.get(cur);
            } while (cur != null);
            cycle.add(start);
            return cycle;
      }



}

package A10_DijkstraPQShortestPath;

import java.util.List;
```

```
public class DijkstraPQShortestPath extends FindWay {
	private int[] dist;

	public DijkstraPQShortestPath(Graph graph) {
		super(graph);
	}

	/**
	 * Startentfernung initialisieren
	 *
	 * @param from
	 *            Startknoten
	 */
	protected void initPathSearch() {
		int numv = graph.numVertices();
		dist = new int[numv];
		for (int i = 0; i < numv; i++) {
			dist[i] = 9999; // Summen im Graph d�rfen nie mehr ergeben
		}
	}

	/**
	 * Berechnet *alle* k?rzesten Wege ausgehend vom Startknoten Setzt dist[]-
	 * und pred[]-Arrays, kein R?ckgabewert
	 *
	 * @param from
	 *            Startknoten
	 */
	protected boolean calculatePath(int from, int to) {

		VertexHeap heap = new VertexHeap(graph.numVertices());
		pred = new int[graph.numVertices()];
		dist[from] = 0;
		for (int i = 0; i < dist.length; i++) {
			Vertex v = new Vertex(i, dist[i]);
			heap.insert(v);
			pred[i] = -1;
		}
		heap.setCost(from, 0);

		while (!heap.isEmpty()) {
			Vertex current = heap.remove();
			List<WeightedEdge> nachbarn = graph.getEdges(current.vertex);
			for (WeightedEdge nachbar : nachbarn) {
				int distToHere = dist[current.vertex];
				int distToNext = nachbar.weight;
				int distInsg = distToHere + distToNext;
				if (distInsg < dist[nachbar.to_vertex]) {
					dist[nachbar.to_vertex] = distInsg;
					heap.setCost(nachbar.to_vertex, distInsg);
					pred[nachbar.to_vertex] = current.vertex;
				}
			}
		}
		return true;
	}
}
```

```
package A11_DijkstraDGShortestPath;
import java.util.List;

public class DijkstraDGShortestPath extends FindWay {
	private int[] dist;
	private boolean[] visited;

	public DijkstraDGShortestPath(Graph graph) {
		super(graph);
	}

	/**
	 * Startentfernung initialisieren
	 */
	protected void initPathSearch() {
		int numv = graph.numVertices();
		dist = new int[numv];
		visited = new boolean[numv];
		for (int i=0; i < numv; i++) {
			dist[i] = 9999;		// Summen im Graph d?rfen nie mehr
ergeben
		}
	}

	/**
	 * Berechnet *alle* k?rzesten Wege ausgehend vom Startknoten
	 * @param from Startknoten
	 */
	protected boolean calculatePath(int from, int to, boolean useChargeRoads) {
		dist[from] = 0;
		while (true) {
			int v = findNearestUnvisitedVertex();
			if (v == -1) {
				return (dist[to] != 9999);
			}
			visited[v] = true;
			List<WeightedEdge> edges = graph.getEdges(v);
			for (WeightedEdge we: edges) {

				if(!useChargeRoads && we.charge)
					continue;

				int newcost = dist[v] + we.weight;
				int tov = we.to_vertex;
				if (newcost < dist[tov]) {
					dist[tov] = newcost;
					pred[tov] = v;
				}
			}
		}
	}

	private int findNearestUnvisitedVertex() {
		int min = 9999;
		int min_pos = -1;
		for (int i=0; i < dist.length; i++) {
			if (dist[i] < min && !visited[i]) {
				min_pos = i;
				min = dist[i];
```

```
                }
            }
            return min_pos;
      }
}

package A12_DijkstraLand;

import java.util.ArrayList;
import java.util.List;

public class Dijkstra {

      public static List<Integer> dijkstra(Graph g, int von, int nach) {

            int[] pred = new int[g.numVertices()];
            int[] dist = new int[g.numVertices()];

            VertexHeap heap = new VertexHeap(g.numVertices());
            for(int i=0; i<dist.length; i++) {
                  dist[i] = 99999;
                  heap.insert(new WeightedEdge(i, 99999));
                  pred[i] = -1;
            }

            dist[von] = 0;
            heap.setPriority(0, 0);

            while(!heap.isEmpty()) {

                  WeightedEdge cur = heap.remove();

                  if (cur.vertex == nach) break;

                  List<WeightedEdge> nachbarn = g.getEdges(cur.vertex);

                  for(WeightedEdge nachbar: nachbarn) {
                        int distBisHier = cur.weight;
                        int distZumNachbar = nachbar.weight;

                        int distInsg = distBisHier + distZumNachbar;

                        if (!
g.getLand(cur.vertex).equals(g.getLand(nachbar.vertex))) {
                              distInsg++;
                        }

                        if (distInsg < dist[nachbar.vertex] ) {

                              dist[nachbar.vertex] = distInsg;
                              heap.setPriority(nachbar.vertex, distInsg);

                              pred[nachbar.vertex] = cur.vertex;
                        }
                  }
            }

            // pred ausgeben
            for(int i=0; i<pred.length; i++) {
```

```
                  System.out.println(i + " ?ber " + pred[i]);
            }


            // Way ausgeben
            System.out.println();
            ArrayList<Integer> way = predToWay(pred, von, nach);
            return way;
      }

      private static ArrayList<Integer> predToWay(int[] pred, int from, int to) {

            ArrayList<Integer> way = new ArrayList<Integer>();

            int i = to;
            while (i != from) {
                  way.add(0, i);
                  i = pred[i];
            }
            way.add(0, from);

            return way;
      }


}
```

```
package A13_MergeSort;

import java.util.Arrays;



public class MergeSort implements PersonenSort {

      /**
       * Sortier-Funktion
       */
      public void sort(Person[] personen) {
            // Start des rekursiven Aufrufs
            sort(personen, 0, personen.length-1);
      }

      /**
       * Rekursive Funktion zum Sortieren eines Teils des Arrays
       * @param personen Zu sortierendes Array
       * @param start    Startpunkt im Array
       * @param end      Endpunkt im Array
       */
      public void sort(Person[] personen, int start, int end)
      {
            if (start >= end) {           // Arrays der L?nge 1 sind sortiert
                  return;
            }

            int mitte = start + ((end-start)/2);

            // Rekursiver Aufruf: Linke & rechte H?lfte sortieren
            sort(personen, start, mitte);
```

```
            sort(personen, mitte+1, end);
            // Nach Rekursion: Linke und rechte H?lfte sind nun sortiert

            // F?r Merge: H?lften in eigene Arrays kopieren
            // Hinweis: bei copyOfRange ist Obergrenze exklusiv, deshalb "+ 1"
            Person[] teil1 = Arrays.copyOfRange(personen, start, mitte+1);
            Person[] teil2 = Arrays.copyOfRange(personen, mitte+1, end+1);
            // Beide H?lften zusammenf?gen und in data-Array schreiben
            merge(teil1, teil2, personen, start);
      }

      /**
       * Merge zweier Arrays in ein Ergebnis-Array
       * @param pers1 Erstes Array
       * @param pers2 Zweites Array
       * @param result Ergebnisarray
       * @param start  Position f?r Speicherung in Ergebnisarray
       */
      public void merge(Person[] pers1, Person[] pers2, Person[] result, int start)
{
            int i = 0;
            int j = 0;
            int next = start;

            // Schleife: Vorderste Elemente vergleichen und in Ergebnisarray einf?
gen
            while (i < pers1.length && j < pers2.length) {
                  if (pers1[i].compareTo(pers2[j]) < 0) {
                        result[next] = pers1[i];
                        i++;
                  }
                  else {
                        result[next] = pers2[j];
                        j++;
                  }
                  next++;
            }

            // Nur einer der beiden nachfolgenden F?lle trifft zu.

            // Falls Array 1 noch ?berbleibsel hat, jetzt anf?gen.
            // (Alle Elemente sind gr??er als das gr??te Element in Array 2)
            for (int k = i; k < pers1.length; k++, next++) {
                  result[next] = pers1[k];
            }

            // Falls Array 2 noch ?berbleibsel hat, jetzt anf?gen.
            // (Alle Elemente sind gr??er als das gr??te Element in Array 1)
            for (int k = j; k < pers2.length; k++, next++) {
                  result[next] = pers2[k];
            }

      }
}
```